

INTRODUCTION TO REST API (Pg.1)

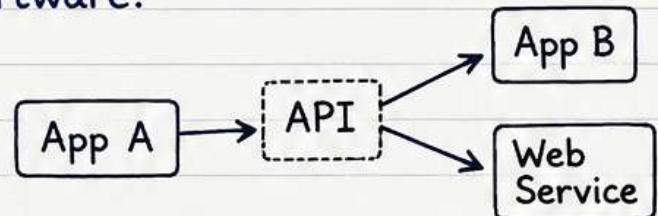
① WHAT IS AN API ?

A set of protocols & tools that allow different applications to communicate with each other.

A "contract" between software.

✦ Functions:

- * Expose data,
- * Trigger actions,
- * Integrate systems.



- ### ✦ Popular Examples:
- Google Maps API, Weather API, Payment gateway API (Stripe), Social media API (Facebook Graph)

② WHAT IS REST ? (REPRESENTATIONAL STATE TRANSFER)

↳ An architectural style defined by Roy Fielding in his 2000 PhD dissertation. NOT a protocol.

- * Uses standard HTTP protocol.
- * Focuses on resources, simplicity & scalability.

③ REST API vs REST

REST

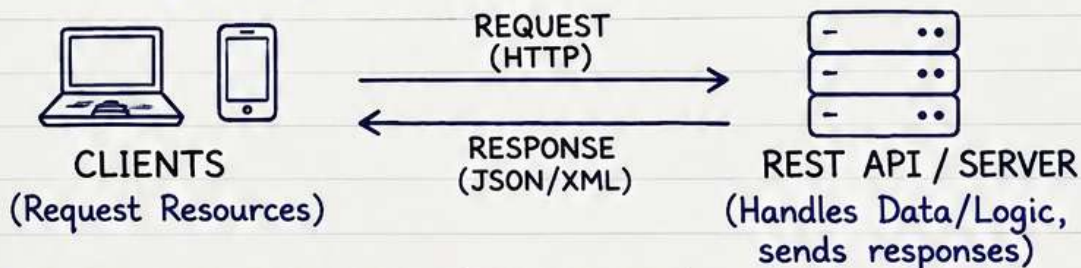
- * An architectural style
- * Set of constraints
- * Independent of any protocol
- * Describes how to design networked applications

REST API

- * An API that follows REST style & constraints
- * Uses HTTP protocol
- * Implements REST principles to build web services

④ CORE CONCEPT : CLIENT-SERVER MODEL

↳ Principle: Separation of Concerns



⑤ REST CONSTRAINTS (PRINCIPLES)

- | | |
|-----------------|-----------------------------|
| * Client-Server | * Uniform Interface |
| * Stateless | * Layered System |
| * Cacheable | * Code on Demand (optional) |

✦ Simple, Scalable, Stateless, Standard.
That's the power of REST! ✦

INTRODUCTION TO REST API (Pg.2)

⑥ HTTP METHODS (VERBS)

↳ HTTP methods define the type of action to be performed on a resource.

METHOD	PURPOSE	EXAMPLE ENDPOINT	IDEMPOTENT ?
GET	Retrieve data	GET /users	Yes
POST	Create new resource	POST /users	No
PUT	Replace entire resource	PUT /users/1	Yes
PATCH	Update part of resource	PATCH /users/1	No*
DELETE	Remove resource	DELETE /users/1	Yes

* PATCH is not always idempotent depending on how it's implemented.

⑦ REST vs REST API

REST (Architectural Style)

- A set of architectural principles.
- Describes constraints & how systems should be designed.
- Protocol independent.

≠

REST API (Implementation)

- An API that follows REST principles.
- Uses HTTP protocol.
- Endpoints, requests, responses etc.
- What we actually build & use.

⑧ RESOURCE & URI (UNIFORM INTERFACE)

↳ In REST, everything is a resource.

↳ Resources are identified by URIs (Uniform Resource Identifiers).

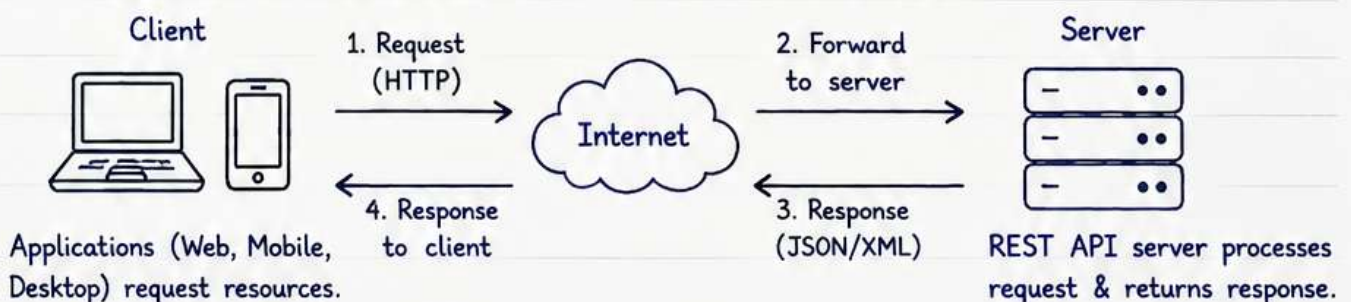
Examples :

- All users → GET /users
- Single user → GET /users/1
- User's posts → GET /users/1/posts
- Create user → POST /users
- Update user → PUT /users/1
- Delete user → DELETE /users/1

Good URI Practices

- ✓ Use nouns, not verbs.
- ✓ Use lowercase letters.
- ✓ Use hyphens for readability.
- ✓ Be consistent.

⑨ HOW REST API WORKS ? (HIGH LEVEL FLOW)



✦ Client makes a request → Server processes → Server sends a response. ✦

INTRODUCTION TO REST API (Pg.3)

⑩ ANATOMY OF AN API REQUEST

↳ A request is sent by the client to the server to perform an action on a resource.

① URL / Endpoint

- The address of the resource.

② Path Parameters

- Used to identify a specific resource.

③ Query Parameters

- Optional parameters to filter / sort / paginate data.

④ Headers

- Provide metadata like content type, auth token, accept type etc.

⑤ Body (Payload)

- Data sent to the server.
(Used in POST, PUT, PATCH)

EXAMPLE REQUEST

GET https://api.example.com/users/123?role=admin&page=2

① Base URL ② Resource ③ Path Param Query Params

Headers:

④ Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR...

Body (JSON):

⑤ {
 "name": "Abhi",
 "email": "abhi@example.com",
 "role": "admin"
}

★ GET requests usually do not have a body.
Body is used in POST, PUT, PATCH requests.

⑪ ANATOMY OF AN API RESPONSE

↳ The server processes the request and sends back a response.

EXAMPLE RESPONSE

Status Line: 200 OK

Headers:

Content-Type: application/json
Date: Tue, 16 Jul 2024 10:30:00 GMT
Server: nginx
Cache-Control: no-cache

Body (JSON):

```
{
  "id": 123,
  "name": "Abhi",
  "email": "abhi@example.com",
  "role": "admin"
}
```

① Status Code

- Indicates the result of the request.
(Success / Redirect / Client Error / Server Error)

② Headers

- Provide metadata about the response
(content type, caching, server info etc.)

③ Body

- Contains the actual data.
- Usually in JSON or XML format.
- Can be empty in some cases.



TIP:

- ✓ Understand the request structure deeply.
- ✓ Always check status code before looking at data.
- ✓ APIs are all about resources, not actions!



INTRODUCTION TO REST API (Pg.4)

12) HTTP STATUS CODES (COMMON)

↳ Status codes help understand the result of a request quickly.

CATEGORY	RANGE	MEANING	EXAMPLES
2xx	200 - 299	Success	200 OK, 201 Created, 204 No Content
3xx	300 - 399	Redirection	301 Moved Permanently, 304 Not Modified
4xx	400 - 499	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	500 - 599	Server Error	500 Internal Server Error, 503 Service Unavailable

★ Always handle errors gracefully in your application.

13) DATA FORMATS USED IN REST API

APIs can return data in different formats. JSON is the most popular.

- JSON → Lightweight, easy to read & parse (Most used)
- XML → Older format, heavier than JSON
- YAML → Human readable, used in some APIs
- Plain Text / CSV → Used in some specific cases

Example (JSON)

```
{
  "id": 1,
  "name": "Abhi",
  "email": "abhi@example.com",
  "role": "admin"
}
```

14) BENEFITS OF REST API

-  1. Platform Independent
Works across different platforms and languages.
-  2. Scalability
Easier to scale because of stateless architecture.
-  3. Performance
Uses HTTP methods, caching and efficient data formats like JSON.
-  4. Simplicity
Easy to understand and implement.
-  5. Flexibility
Can be used by web, mobile, desktop and third-party systems.



KEY TAKEAWAY

- ✓ REST API is about resources, not actions.
- ✓ Use proper HTTP methods.
- ✓ Follow good URI structure.
- ✓ Always return meaningful status codes.
- ✓ Keep APIs simple, consistent and well documented.

✧ Well designed APIs make systems scalable, maintainable & reliable! ✧

★ Understand the basics deeply. That's how great APIs are built! ★

INTRODUCTION TO REST API (Pg.5)

15) RESTFUL RESOURCE DESIGN

↳ Good API design makes your API easy to use and understand.

↳ Everything in REST is a resource.

1) RESOURCE = NOUN, NOT VERB

- Use nouns to represent resources.
- Avoid verbs in URL.

✓ Good : /users
/orders
/products

✗ Bad : /getUsers
/createOrder
/deleteProduct

2) BASE URI

- Keep a consistent base URL for all resources.
- Example:

https://api.example.com/v1
 Protocol Domain Version

(v1 is the API version)

3) RESOURCE HIERARCHY (NESTED RESOURCES)

- Show relationship between resources.
- Use nesting only when necessary.

Examples :

- All users → GET /users
- Single user → GET /users/1
- User's posts → GET /users/1/posts
- Single post → GET /users/1/posts/10
- Comments on a post → GET /users/1/posts/10/comments

☆ Don't go too deep.
More than 2-3 levels
becomes hard to manage.
Keep it simple!

4) FILTERING, SORTING & SEARCH

- Help clients get exactly what they need.
- Use query parameters.

Examples :

/users?role=admin (filter)
/users?sort=name (sort)
/users?search=abhi (search)
/users?page=2&limit=10 (pagination)

5) PAGINATION

- Don't return huge data in one response.
- Use page & limit (or per_page).

Example :

GET /users?page=2&limit=10
 ↳ page = 2 (which page)
 ↳ limit = 10 (items per page)

Response also includes total count or next page info.

6) URI NAMING BEST PRACTICES

- ✓ Use lowercase letters.
- ✓ Use hyphens (-) for readability.
- ✓ Use plural nouns.
- ✓ Keep it short and meaningful.
- ✓ Be consistent across your API.

7) EXAMPLE: GOOD URI DESIGN

Users → /users
 User by ID → /users/1
 User's Orders → /users/1/orders
 Order by ID → /orders/55
 Order Items → /orders/55/items
 Products → /products
 Product by ID → /products/99
 Product Reviews → /products/99/reviews



TIP:

- ☆ Design your API for the developer who will use it.
- ☆ A well designed API is easy to learn and hard to misuse.

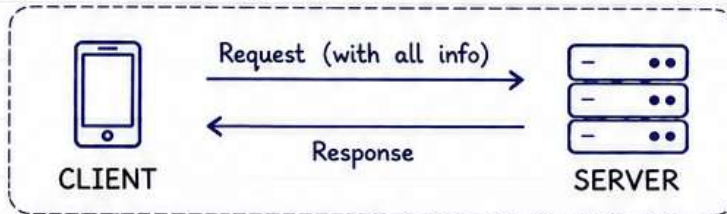


INTRODUCTION TO REST API (Pg.6)

16 STATELESSNESS IN REST API

↳ REST APIs are stateless.

↳ Each request from the client must contain all the information needed to understand and process the request.



☆ WHY STATELESS ?

- * Easier to scale
- * No server side session
- * More reliable
- * Better performance

17 IDEMPOTENCY IN REST API

↳ Multiple identical requests have the same effect as a single request.

METHOD	IDEMPOTENT ?	WHY ?	EXAMPLE
GET	YES	Retrieves data. No change in resource.	GET /users/1 (Called once or many times → same result)
POST	NO	Creates a new resource every time.	POST /users (Each call creates a new user)
PUT	YES	Replaces the entire resource. Same data → same state.	PUT /users/1 (Replace user 1 every time)
PATCH	NO*	Partial update. Not guaranteed idempotent in all cases.	PATCH /users/1 (Depends on how it's used)
DELETE	YES	Deletes the resource. If already deleted, same result.	DELETE /users/1 (Deleting again → still deleted)

☆ Idempotency is important for safe retries in case of network failures.

18 SAFE METHODS vs IDEMPOTENT METHODS

SAFE METHODS

- Do not modify the resource on the server.
- Read-only operations.
- Example: GET, HEAD, OPTIONS, TRACE

Safe means "no change on server".
All safe methods are idempotent.

IDEMPOTENT METHODS

- Multiple identical requests have the same effect as a single request.
- Resource state remains same.
- Example: GET, PUT, DELETE, HEAD, OPTIONS

Idempotent means "same result".
Not all idempotent methods are safe (e.g., PUT, DELETE can change data).

19 COMMON STATUS CODES QUICK REFERENCE

200	OK	Request successful	400	Bad Request	Invalid request
201	Created	Resource created successfully	401	Unauthorized	Authentication required
204	No Content	Success, but no content to return	403	Forbidden	No permission
301	Moved Permanently	Resource moved permanently	404	Not Found	Resource not found
304	Not Modified	Use cached version	500	Internal Server Error	Server-side error

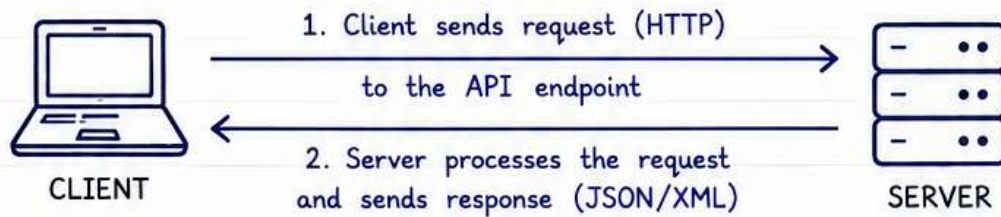


Design simple. Keep it consistent. Follow REST principles.
Your API will be developer-friendly and future-ready!



INTRODUCTION TO REST API (Pg.7)

20 REQUEST vs RESPONSE FLOW (SUMMARY)



☆ This request-response cycle happens every time you call an API.

21 REAL LIFE EXAMPLE: GET /users/1

REQUEST

GET <https://api.example.com/v1/users/1>

Headers:

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

Accept: application/json

RESPONSE (200 OK)

Status Line: 200 OK

Content-Type: application/json

Body:

```
{
  "id": 1,
  "name": "Abhi",
  "email": "abhi@example.com",
  "role": "admin"
}
```

☆ You asked for user with id = 1, server found it and returned the data.

22 REAL LIFE EXAMPLE: POST /users

REQUEST

POST <https://api.example.com/v1/users>

Headers:

Authorization: Bearer eyJhbGciOiJIUzI1NiIs...

Content-Type: application/json

Body:

```
{
  "name": "Abhi",
  "email": "abhi@example.com",
  "role": "user"
}
```



RESPONSE (201 Created)

Status Line: 201 Created

Content-Type: application/json

Body:

```
{
  "id": 101,
  "name": "Abhi",
  "email": "abhi@example.com",
  "role": "user"
}
```

☆ New user created successfully. Server returns the created resource with 201 status.

23 WHAT MAKES AN API "RESTFUL" ?

- * Uses HTTP methods correctly.
- * Follows resource based URIs.
- * Stateless communication.
- * Returns proper status codes.
- * Uses standard data formats (JSON/XML).
- * Simple, consistent and predictable.



REMEMBER

REST is not a protocol.
It's an architectural style.
It uses HTTP as the underlying protocol.



Understand the fundamentals well.
Strong basics = Confident in interviews & real world!



INTRODUCTION TO REST API (Pg.7)

20 AUTHENTICATION IN REST API

↳ Authentication is used to verify who is making the request.

① API KEY



- Simple & easy
- Sent in header
- Good for public APIs

x-api-key: abc123xyz

② Basic Auth



- Uses username & password
- Encoded in Base64

Authorization:
Basic Zm9vOmJhcG==

③ Bearer Token



- Token based
- Sent in header
- Most common (JWT)

Authorization:
Bearer eyJhbGciOiJI...

④ OAuth 2.0

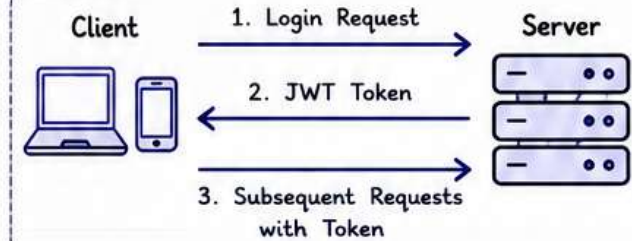


- Authorization framework
- Used by Google, Facebook etc.

21 JWT (JSON WEB TOKEN) - QUICK OVERVIEW

↳ JWT is a popular way to securely transmit information between client and server.

- * Compact
- * Self-contained
- * Secure (when used properly)
- * Stateless



22 SECURITY BEST PRACTICES

✓ Use HTTPS

- Always use HTTPS, not HTTP.

https://

✓ Validate Input

- Always validate user input.
- Prevent injection attacks.

✓ Authorization

- Check user permission before performing action.

✓ Rate Limiting

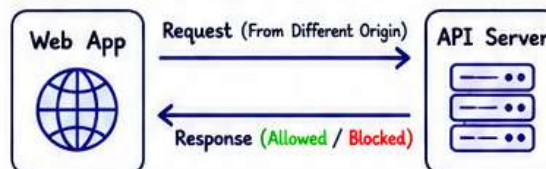
- Limit no. of requests from a single user.

✓ Use Strong Auth

- Prefer JWT / OAuth
- Avoid storing sensitive data in plain text.

23 CORS (CROSS-ORIGIN RESOURCE SHARING)

↳ CORS allows or restricts requests from different origins (domains).



Example

Frontend: app.com
API: api.example.com
CORS decides whether this request is allowed.

24 RATE LIMITING & THROTTLING

↳ Prevent abuse by limiting too many requests.

Example

Limit: 100 requests per minute per user.

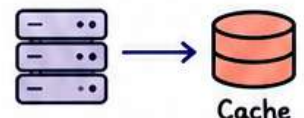


Limit

BONUS: CACHING IN REST API

↳ Reduces server load & improves performance.

- Use Cache-Control header
- Use ETag / Last-Modified
- Helps in faster response



TIP:

- ✓ Secure your APIs from day one!
- ✓ Always think about performance, security & usability.
- ✓ A good API is reliable, secure, fast & easy to use.



INTRODUCTION TO REST API (Pg.8)

24 ERROR HANDLING IN REST API

- ↳ Always return meaningful error messages.
- ↳ Use proper status codes and consistent structure.

EXAMPLE ERROR RESPONSE (400 Bad Request)

```
{
  "status": 400,
  "error": "Bad Request",
  "message": "Email is required",
  "path": "/v1/users",
  "timestamp": "2025-05-20T10:30:00Z"
}
```

COMMON ERRORS

- 400 Bad Request → Invalid input
- 401 Unauthorized → Authentication required
- 403 Forbidden → No permission
- 404 Not Found → Resource not found
- 409 Conflict → Resource conflict
- 500 Internal Server Error → Server issue
- 503 Service Unavailable → Server overloaded



Good error messages save time for developers and users.

25 VERSIONING IN REST API

- ↳ APIs evolve. Versioning helps manage changes without breaking existing clients.
- ↳ Common way: include version in the URL.

EXAMPLES

- <https://api.example.com/v1/users>
- <https://api.example.com/v2/users>
- <https://api.example.com/v1/products>

BEST PRACTICES

- ✓ Start with v1 from day one.
- ✓ Support old versions for some time.
- ✓ Communicate deprecation in advance.
- ✓ Don't make breaking changes in same version.

26 RATE LIMITING

- ↳ Prevent abuse and ensure fair usage.
- ↳ Limit the number of requests a client can make in a given time.

EXAMPLE HEADERS

- X-RateLimit-Limit: 1000
- X-RateLimit-Remaining: 250
- X-RateLimit-Reset: 1716182400
(time when limit resets)



WHY RATE LIMITING ?

- Protects server from overload
- Prevents brute force attacks
- Ensures fair usage for all
- Helps in cost control

27 TOOLS TO TEST REST API



POSTMAN

Most popular tool for testing APIs. Easy to use & powerful.



INSOMNIA

Clean UI, fast and developer friendly.



THUNDER CLIENT

VS Code extension. Simple & lightweight.



CURL

Command line tool. Great for scripting and automation.



REST API is all about resources, proper methods, stateless communication and good design.
Master the basics and you're halfway there!



INTRODUCTION TO REST API (Pg. 9)

28 REST API BEST PRACTICES

1. Keep it Simple

- Follow REST principles.
- Don't over-complicate the API.

★ Simple APIs are easy to build, use and maintain.

3. Use Proper HTTP Methods

- GET → Read data
- POST → Create data
- PUT → Replace data
- PATCH → Partial update
- DELETE → Delete data

★ Use the right method for the right action.

5. Meaningful Status Codes

- Use appropriate HTTP status codes.

Common Status Codes

200	OK	→	Success
201	Created	→	Resource created
400	Bad Request	→	Invalid input
401	Unauthorized	→	Authentication required
403	Forbidden	→	No permission
404	Not Found	→	Resource not found
500	Server Error	→	Server side error

7. Filtering, Sorting & Search

- Help clients find exactly what they need.
- Use query parameters.

Example:

/users?role=admin (filter)
 /users?sort=name (sort)
 /users?search=abhi (search)
 /users?page=2&limit=10 (pagination)

9. Secure Your API

- ✓ Use HTTPS
- ✓ Authenticate & authorize requests
- ✓ Validate & sanitize user input
- ✓ Protect against common attacks (SQLi, XSS, etc.)

2. Use Nouns, Not Verbs

- Use nouns for resources.
- Avoid verbs in the URL.

✓ Good : /users
/orders/123

✗ Bad : /getUsers
/createOrder

4. Consistent Response Format

- Always return responses in JSON (or XML).
- Keep the structure consistent across all APIs.

```
{
  "success": true,
  "data": { ... },
  "message": "User fetched successfully"
}
```

6. Pagination for Large Data

- Don't return huge data in a single response.
- Use page and limit (or per_page).

Example:

GET /users?page=2&limit=10
 ↳ page = 2 (which page)
 ↳ limit = 10 (items per page)

8. Validation & Error Handling

- Validate input on both client and server.
- Return meaningful error messages.

Example Error Response (400 Bad Request)

```
{
  "success": false,
  "message": "Email is required",
  "errors": {"email": "Email cannot be empty"}
}
```

10. Document Your API

- ✓ Good documentation saves time.
- ✓ Use tools like Swagger / OpenAPI.
- ✓ Keep docs up to date with changes.

API
DOCS

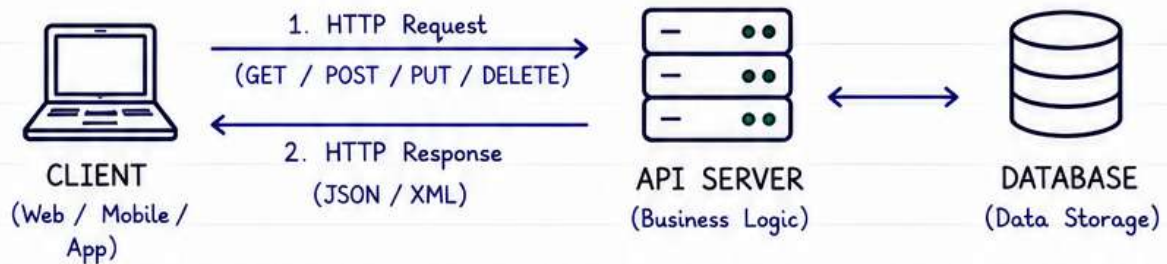


Good design + Consistency + Security = Great REST API
Build APIs that developers love to use!



INTRODUCTION TO REST API (Pg.10)

29 REST API ARCHITECTURE OVERVIEW



★ Client sends request → API Server processes → Database interaction (if needed) → Response sent back to Client.

30 REST API vs SOAP API

FEATURE	REST API (Representational State Transfer)	SOAP API (Simple Object Access Protocol)
Protocol	Uses HTTP	Uses HTTP / SMTP / Others
Data Format	JSON, XML, Text, etc.	Only XML
Performance	Faster and lightweight	Slower and heavy
Ease of Use	Easy to use and understand	Complex (lots of standards)
State	Stateless	Can be Stateful
Caching	Supports caching	Does not support caching
Best For	Web, Mobile, Microservices	Enterprise applications

31 HTTP METHODS QUICK RECAP

GET	Retrieve data. Should not change anything on server.	Example: GET /users/1
POST	Create new resource.	Example: POST /users
PUT	Replace entire resource with new data.	Example: PUT /users/1
PATCH	Partially update a resource.	Example: PATCH /users/1
DELETE	Delete a resource.	Example: DELETE /users/1

32 COMMON REST API ENDPOINT PATTERNS

• GET /users → Get all users	• DELETE /users/{id} → Delete user
• GET /users/{id} → Get user by id	• GET /users/{id}/orders → Get user orders
• POST /users → Create new user	• POST /users/{id}/orders → Create order for user
• PUT /users/{id} → Update user (full)	• GET /orders/{id} → Get order by id
• PATCH /users/{id} → Partially update user	• DELETE /orders/{id} → Delete order by id

33 PRACTICE TIPS



- ✓ Start by building small APIs.
- ✓ Test using Postman / Insomnia.
- ✓ Understand request & response deeply.
- ✓ Read API docs before using any API.

★ The more you practice, the better you'll design and use APIs.



REST API is the backbone of modern applications.
Learn it well, build it better, use it everywhere!

